

ENFORCING DOCKER NETWORKING

Let us look at a scenario where a **fintech company** wants you to design the Docker network for three of its containers.

This company wants its **web application** and **database** on the **same network**, but wants its **finance payments** on **another network totally isolated from the network** of its **web application and database**.

Because **payment data is extremely sensitive** and needs to be isolated from everything else, if all containers share the same network, any container that gets compromised can potentially **see and intercept traffic** from every other container, including payment data.

A dedicated network means only containers that **explicitly need to talk to the payment container** are allowed to.

REAL REASONS WHY THE FINTECH COMPANY WANTS AN ISOLATED NETWORK FOR IT FINANCE PAYMENTS.

I) SECURITY

- Payment containers handle card numbers, bank details, transaction data
- Keeping them on a separate network limits who can reach them
- A hacked marketing container cannot sniff payment traffic; it cannot see

II) COMPLIANCE WITH PCI DSS

- Any organization handling card payments must comply with **PCI DSS** (Payment Card Industry Data Security Standard)
- PCI DSS explicitly requires payment systems to be on an **isolated, controlled network segment**
- Failing this means failing your audit

III) BLAST RADIUS REDUCTION

- If something goes wrong elsewhere in your infrastructure, the payment network is unaffected
- The damage stays contained to where it happened

IV) TRAFFIC CONTROL

- Only specific containers (checkout service, order service) are allowed on the payment network
- Everything else, like frontend, database, admin panel, has zero access

To achieve this, we can use a **bridge Docker network** type.

Alternatively, **Docker Compose** would allow you to **manage** and **design the Docker networks of these different containers**, but to see how **Docker networking works at the basics**, we will use **Docker commands to effect this**.

STEPS

1) ON YOUR LOCAL MACHINE, START THE DOCKER ENGINE, AND OPEN WSL. CONFIRM DOCKER IS RUNNING BY TYPING THE COMMAND BELOW.

```
docker --version
```

this shows the current running version of Docker

2) PULL AN NGINX IMAGE FROM DOCKER HUB FOR THIS PRACTICE, GIVE THE CONTAINER A CUSTOMIZED NAME, AND RUN THAT CONTAINER IN DETACHED MODE (-d).

TYPE THE COMMAND BELOW.

```
docker run -d --name web-app nginx:latest
```

this pulls the nginx image from Docker Hub

and gives it a customized name called

web-app instead of any random name

3) CREATE AN INTERACTIVE SESSION INSIDE THE WEB APP NGINX RUNNING CONTAINER TO ALLOW YOU TO RUN BASH COMMANDS.

TYPE THE COMMANDS BELOW.

```
docker exec -it web-app /bin/bash
```

this creates an interactive bash environment

from the running web-app container

```
apt update # update your server
```

```
apt-get install iputils-ping -y
```

```
ping -V # confirm you have ping installed
```

4) OPEN ANOTHER TERMINAL AND CREATE ANOTHER RUNNING CONTAINER FROM THE ALREADY INSTALLED NGINX IMAGE. GIVE THE CONTAINER THE NAME `database`.

TYPE THE COMMAND BELOW.

```
docker run -d --name database nginx:latest      # create a running container called database
```

```
docker ps      # list all the running containers you have running. You should see the
               # ids of the web-app and database running containers.
```

5) INSPECT THE DEFAULT BRIDGE DOCKER NETWORK OF YOUR RUNNING CONTAINERS (web-app and database).

TYPE THE COMMAND BELOW.

```
docker inspect web-app
```

locate network settings in your JSON result.

observe the IP address for your web-app is 172.17.0.2

and under Networks, you have a name bridge.

docker inspect database

locate network settings in your JSON result.

observe the IP address for your web-app is 172.17.0.3

and under Networks, you have a name bridge.

172.17.0.2 AND 172.17.0.3 SHOW THAT THE web-app AND database CONTAINERS BELONG TO THE SAME NETWORK, WHICH MAKES THEM ABLE TO COMMUNICATE.

TO CONFIRM THIS, WE ARE GOING TO PING THE IP ADDRESS (172.17.0.3) OF THE database CONTAINER FROM THE INTERACTIVE TERMINAL OF THE web-app CONTAINER OPENED IN THE FIRST TERMINAL.

SWITCH BACK TO THE FIRST TERMINAL AND TYPE THE COMMAND BELOW.

ping 172.17.0.3

if this PINGS, the web-app container can communicate with

the database container because they are on the same

network. This is the default bridge networking in action.

SEE ALL NETWORKS CURRENTLY RUNNING ON YOUR DOCKER ENVIRONMENT.

SWITCH BACK TO THE SECOND TERMINAL AND TYPE THE COMMAND BELOW.

docker network ls

this shows all the networks currently running on your

docker environment. Observe the bridge network.

6) WE ARE GOING TO CREATE A THIRD CONTAINER, BUT WE DO NOT WANT THIS CONTAINER TO BE IN THE SAME NETWORK AS THE web-app AND database CONTAINERS BECAUSE THIS CONTAINER HOLDS SENSITIVE INFORMATION.

TO ACHIEVE THIS, WE WILL CREATE A USER-DEFINED BRIDGE NETWORK.

CREATE A USER-DEFINED BRIDGE NETWORK BY SPECIFYING THE NAME OF THAT NETWORK.
IN THE SECOND TERMINAL, TYPE THE COMMAND BELOW.

`docker network create <your-network-name>`

`docker network create finance-network` **# this creates a network called**
finance-network

`docker network ls` **# this shows the finance-network**
created as a customized bridge
network type

create a finance-payments running container from the nginx pulled image and attach
the finance-network to the running finance-payments container

`docker run -d --name <your-container-name> --network=<your-network-name> nginx:latest`

`docker run -d --name finance-payments --network=finance-network nginx:latest`

`docker ps` **# this will show the finance-payments, web-app, and databases**
containers running in the background

`docker inspect finance-payments` **# locate network settings in your JSON result.**
under Networks, you have a name called
finance-networks. Also observe the IP address for

your finance-payments is "172.18.0.2"; this is different
from the web-app and database IP addresses.
this means that finance-payments container
is not in the same network as web-app and database
containers

172.17.0.2 AND 172.17.0.3 SHOW THAT THE web-app AND database CONTAINERS BELONG TO THE SAME NETWORK, WHICH MAKES THEM ABLE TO COMMUNICATE.

BUT 172.18.0.2 SHOWS THAT THE finance-payments CONTAINER BELONGS TO A SEPARATE NETWORK AND CANNOT COMMUNICATE WITH THE web-app AND database CONTAINERS.

TO CONFIRM THIS, WE ARE GOING TO PING THE IP ADDRESS (172.18.0.2) OF THE finance-payments CONTAINER FROM THE INTERACTIVE TERMINAL OF THE web-app CONTAINER OPENED IN THE FIRST TERMINAL.

SWITCH BACK TO THE FIRST TERMINAL AND TYPE THE COMMAND BELOW.

ping 172.18.0.2 # if it does not PING, the web-app container cannot
 # communicate with the finance-payments container
 # because they are on different network paths. This also
 # applies to the database container.

7) WE ARE GOING TO CREATE A FOURTH CONTAINER CALLED host-resource, BUT THIS TIME, WE WILL USE A HOST DOCKER NETWORK TYPE.

SWITCH BACK TO THE SECOND TERMINAL AND TYPE THE COMMANDS BELOW.

create a host-resource running container from the nginx pulled image and use
the host docker network type on the running host-resource container

```
docker run -d --name <your-container-name> --network=host nginx:latest
```

```
docker run -d --name host-resource --network=host nginx:latest
```

```
docker ps
```

**# this will show the host-resource, finance-payments, web-app,
and databases containers running in the background**

```
docker inspect host-resource
```

**# locate network settings in your JSON result.
under Networks, you have a name called
hosts. Also observe the IP address for
your host-resource container is empty “ ”.
this means that the host-resource container
uses the network identity of the host machine
and has no network identity of its own,
unlike a bridge Docker network.**

**BECAUSE THE host-resource CONTAINER DOES NOT HAVE A NETWORK INDEPENDENTLY, YOU
WILL NEED TO ACCESS THE CONTAINER FROM A HOST MACHINE THAT THE CONTAINER IS
RUNNING ON, WHICH MIGHT BE AN EC2 INSTANCE OR ON-PREM INFRASTRUCTURE.**

8) STOP AND REMOVE YOUR CONTAINERS AND IMAGE.

TYPE THE COMMANDS BELOW.

```
docker ps -a
```

show all stopped and running containers

```
docker stop <your-container-id>
```

```
docker stop 3d9fa8018372
```

stop all running containers by the container ID

`docker rm <your-container-id>`

`docker rm 3d9fa8018372`

remove all containers by the container ID

`docker images`

list all the images you have

`docker rmi <your-docker-image>`

`docker rmi nginx:latest`

delete the nginx image

`docker ps -a`

confirm that all your containers are deleted

`docker images`

confirm that all your images are deleted

`docker network ls`

check all your docker networks

`docker network rm <your-created-network>`

`docker network rm finance-network`

remove a docker network